

The background features a light blue and white geometric pattern. A prominent starburst or sunburst design is centered at the bottom, with rays extending upwards and outwards. The rays are composed of various shades of blue, green, and yellow. The overall aesthetic is modern and technical.

DATA MINING PROJECT

1. Introduction

This project presents an **Advanced Data Mining Dashboard** designed to perform end-to-end data analysis through an interactive graphical user interface (GUI). The system integrates **data validation, cleaning, feature engineering, dimensionality reduction, clustering, and classification**, enabling users to upload raw CSV datasets and obtain meaningful analytical insights without manual preprocessing.

The solution is implemented in **Python** and combines: - **Gradio** for the user interface - **Pandas & NumPy** for data manipulation - **Scikit-learn** for machine learning and statistical modeling - **Matplotlib & Plotly** for data visualization

The project is structured into modular components: 1. GUI layer (`gui_app.py`) 2. Data mining & unsupervised learning logic (`project1_logic.py`) 3. Predictive modeling & supervised learning logic (`project2_logic.py`)

This modular design improves maintainability, scalability, and clarity of responsibilities.

2. System Architecture Overview

The overall workflow of the system is as follows:

1. User uploads a CSV file via the GUI
2. Dataset is passed to the processing logic
3. Automated data cleaning and validation are applied
4. Feature engineering and transformations are performed
5. Machine learning models are executed
6. Visualizations, logs, and processed data are returned to the GUI

Each processing stage is logged to enhance transparency and traceability.

3. Graphical User Interface (GUI) Implementation

3.1 Purpose

The GUI provides an intuitive interface that allows non-technical users to run advanced data mining tasks without writing code.

3.2 Core Responsibilities

- Accept CSV file uploads
- Trigger analysis pipelines
- Display logs, plots, and processed datasets

3.3 Core Logic Explanation

File Processing Function:

```
result = run_project1(df)
```

This line delegates all analytical logic to the backend module, enforcing separation of concerns.

Visualization Rendering:

```
pca_plot = px.scatter(df, x="PC1", y="PC2", color="Cluster")
```

This creates an interactive PCA scatter plot, allowing users to visually inspect clustering results.

Event Binding:

```
run_btn.click(process_file, inputs=file_input, outputs=[...])
```

This connects user actions to backend execution and output rendering.

4. Project 1: Data Cleaning, PCA, and Clustering

4.1 Objective

The goal of Project 1 is to clean customer-related data, reduce dimensionality using **Principal Component Analysis (PCA)**, and group records using **K-Means clustering**.

4.2 Data Validation and Preparation

Missing Column Handling:

```
if col not in df.columns:
    df[col] = np.nan
```

Ensures schema consistency and prevents runtime errors.

Type Conversion and Range Validation:

```
df[col] = pd.to_numeric(df[col], errors="coerce")
```

Invalid entries are safely converted to missing values.

Outlier and Invalid Value Handling:

```
df.loc[(df["Age"] <= 0) | (df["Age"] > 90), "Age"] = np.nan
```

This enforces realistic constraints on the data.

Missing Value Imputation:

```
df[col] = df[col].fillna(df[col].median())
```

Median imputation is used to reduce the impact of skewed distributions.

4.3 Feature Encoding

Categorical Encoding:

```
df["Gender"] = df["Gender"].map({"male": 0, "female": 1}).fillna(0)
```

Transforms categorical variables into numerical format suitable for ML algorithms.

4.4 Principal Component Analysis (PCA)

Standardization:

```
X_scaled = StandardScaler().fit_transform(X)
```

Ensures equal contribution of all features.

Dimensionality Reduction:

```
X_pca = PCA(n_components=2).fit_transform(X_scaled)
```

Reduces the dataset to two orthogonal components while preserving maximum variance.

4.5 Clustering with K-Means

```
kmeans = KMeans(n_clusters=3)
df["Cluster"] = kmeans.fit_predict(X_scaled)
```

Groups data points into three distinct clusters based on feature similarity.

5. Project 2: Predictive Modeling with Logistic Regression

5.1 Objective

Project 2 focuses on **binary classification**, predicting whether a transaction amount is above or below the median value.

5.2 Feature Engineering

Item Count Extraction:

```
df["Item_Count"] = df["Items_Purchased"].apply(lambda x: len(x.split(",")))
```

Transforms textual purchase data into a numerical feature.

5.3 Train-Test Splitting

```
train_test_split(X, y, test_size=0.25, stratify=y)
```

Preserves class balance and ensures reliable evaluation.

5.4 Model Training

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Trains a logistic regression classifier optimized for binary outcomes.

5.5 Model Evaluation

Confusion Matrix:

```
ConfusionMatrixDisplay.from_predictions(y_test, model.predict(X_test))
```

Provides an intuitive visualization of classification performance.

Feature Importance:

```
model.coef_
```

Shows the influence of each feature on the prediction.

6. Conclusion

This project delivers a **professional-grade data mining solution** that integrates data preprocessing, unsupervised learning, supervised modeling, and interactive visualization within a unified dashboard.

Key strengths include: - Modular and scalable architecture - Automated data cleaning and validation - Clear separation between UI and logic - Strong use of industry-standard libraries

The system can be extended to support additional datasets, algorithms, and real-world business analytics use cases.

